Boolean Algebra

Computer Organization

Chandra Gummaluru

chandra-gummaluru.github.io



Boolean Algebra

a set of rules for writing and simplifying expressions over binary values

notation

We write the three basic gates as operations on variables:

- $a \cdot b$
  the AND of a and b
- $a + b$
  the OR of a and b
- $\overline{a}$
  the NOT of a, also written a' or $\neg a$

A **Boolean expression** combines these to describe a circuit.

laws

Each law comes in two forms, one for `AND` and one for `OR`.

identity	$\mathtt{a} \cdot 1 = \mathtt{a}$	$\mathtt{a} + 0 = \mathtt{a}$
null	$\mathtt{a} \cdot 0 = 0$	$\mathtt{a} + 1 = 1$
idempotent	$\mathtt{a} \cdot \mathtt{a} = \mathtt{a}$	$\mathtt{a} + \mathtt{a} = \mathtt{a}$
complement	$\mathtt{a} \cdot \overline{\mathtt{a}} = 0$	$\mathtt{a} + \overline{\mathtt{a}} = 1$

Expressions can also be reordered and regrouped.

commutative	$a \cdot b = b \cdot a$
associative	$a \cdot (b \cdot c) = (a \cdot b) \cdot c$
distributive	$a \cdot (b + c) = a \cdot b + a \cdot c$
involution	$\overline{\overline{a}} = a$

Two more laws let us delete terms that add nothing. They are worth memorising, since they remove gates.

absorption $\quad a + a \cdot b = a$

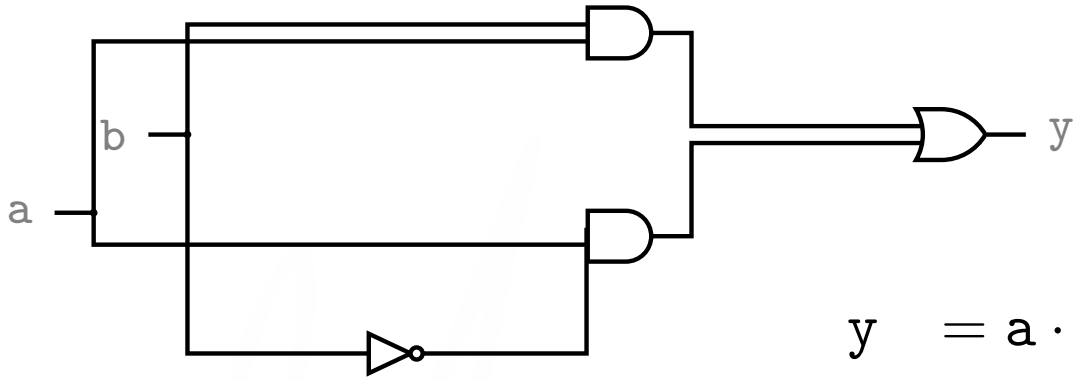
consensus $\quad a \cdot b + \overline{a} \cdot c + b \cdot c = a \cdot b + \overline{a} \cdot c$

De Morgan's laws allow us to flip between ANDs and ORs.

first version $\quad \overline{a \cdot b} = \overline{a} + \overline{b}$

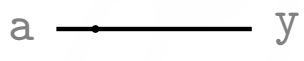
second version $\quad \overline{a + b} = \overline{a} \cdot \overline{b}$

We reduce an expression by applying the laws in turn.



four gates





zero gates

$$
\begin{aligned}
y &= a \cdot b + a \cdot \overline{b} && \\
&= a \cdot (b + \overline{b}) && \text{distributive} \\
&= a \cdot 1 && \text{complement} \\
&= a && \text{identity}
\end{aligned}
$$

simpler expressions mean simpler circuits